MST è un insieme di archi del grafo in input

```
KRUSKAL-MST(V, E, w)
S = ∅
Q = E          /* array Q che contiene gli archi
O(|E| log |E|) Q = SORT(Q)   /* per peso crescente
                             ordino gli archi
for each v ∈ V do
    MAKE-set(v)
for each (u,v) ∈ Q preso nell'ORDINE      *
    if find-set(u) ≠ find-set(v)
        UNION(u, v)
        S = S ∪ {(u,v)}
return S
```

- l'istruzione $Q = \text{SORT}(Q)$
  costo $O(|E| \log |E|)$

- for each $v \in V$ do ?
  MAKE-set(v)

$n = |V|$ # operazioni di MAKE-SET

- # operazioni di find-SET dipende dagli archi in Q
  (for each $(u,v) \in Q$ ...
  find-set ...)

costo *:
$|E| = m$ • operazioni di find-set (2)
+ (1) UNION

N° totale di operazioni su insiemi disjunti =

$= n + O(m) = |V| + O(|E|)$

Se implemento gli INSIEMI DISGIUNTI mediante lungo + compressione

costo per m operazioni su n oggetti $O(m \cdot \alpha(n))$

$\alpha(n)$ quando n è molto grande $\alpha(n) \leq 4$

$\alpha(n)$ funzione che cresce molto lentamente rispetto ad n (ANALISI AMMORTIZZATA)

siccome $\alpha(n) \leq 4$ costante.

$|V| + O(|E|)$ operazioni di union modificate per una costante

$O(|V| + O(|E|)) =$

$O(|V| + |E|)$ costo

che esclude l'operazione

di SORT - per cui somma

$O(|E| \log |E|)$

NB. $|E| \; ? \; |V|$

siccome G il grafo in

input è connesso

$|V| \leq |E| + 1$, perché

$|E| \geq |V| - 1$

si ha che $|V| = O(|E|)$

siccome faccio la somma

$O(|V| + |E|) + O(|E| \log |E|)$

siccome $|V| = O(|E|)$

$O(|E|) + O(|E| \log |E|)$

$\Downarrow$

$O(|E| \log |E|)$

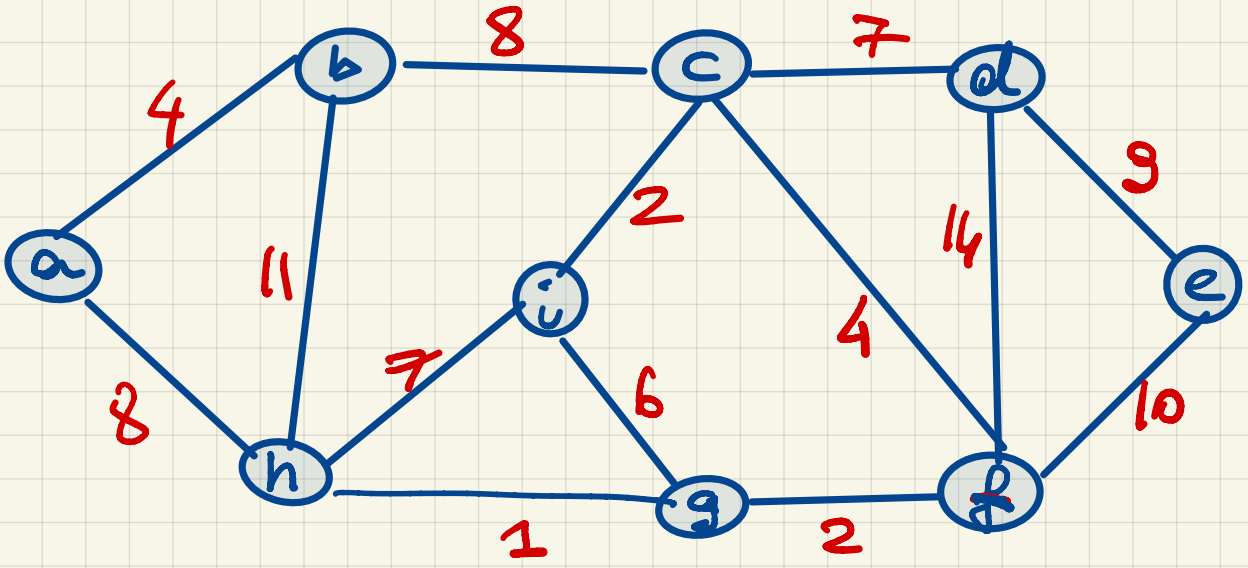
CORMEN

$\log |E| = O(\log |V|)$

conseguenza del fatto che $|E| < |V|^2$

$\Downarrow$

$O(|E| \log |V|)$



osservazione 1:

l'input è un grafo non ORIENTATO!

pesi sono positivi!

Simulazione di KRUSKAL_MST

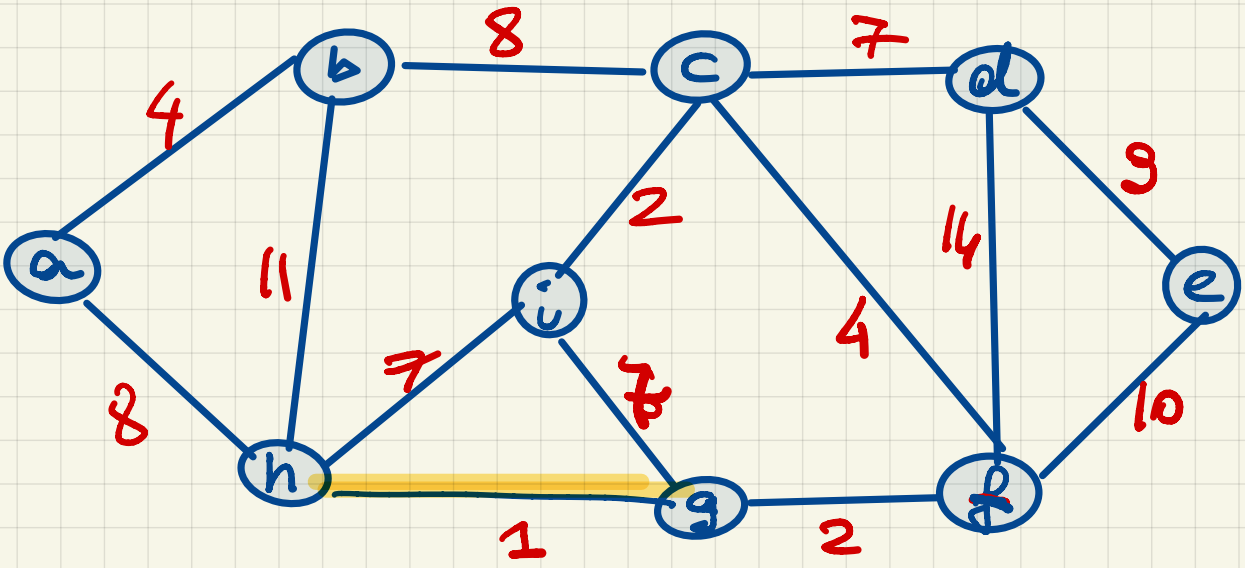
b c d

a i e

h g f

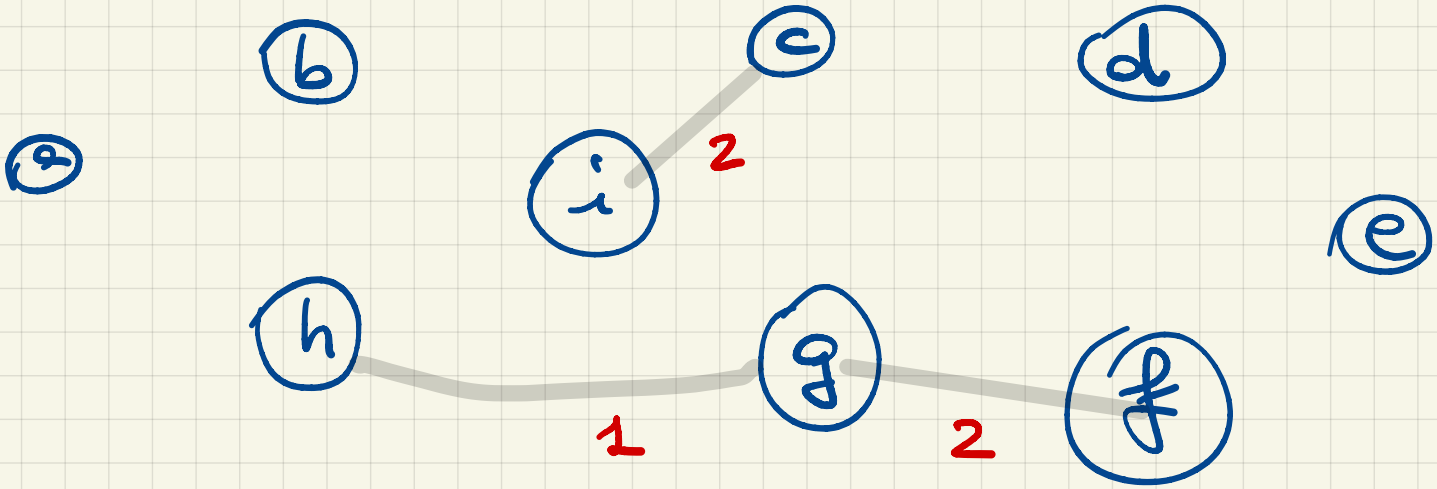
- MAKE_SET(v), $\forall v \in V$

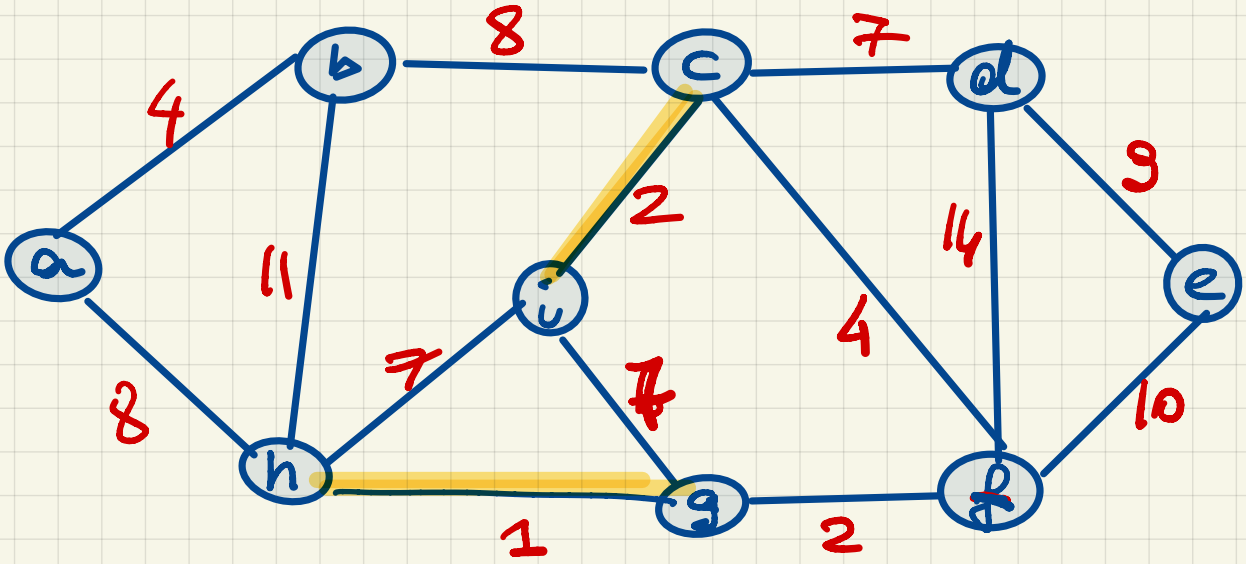
SORT(E) "sottointeso di punto pl' ordi per ordine crescente" -



if find-set(h) ≠ find-set(g)

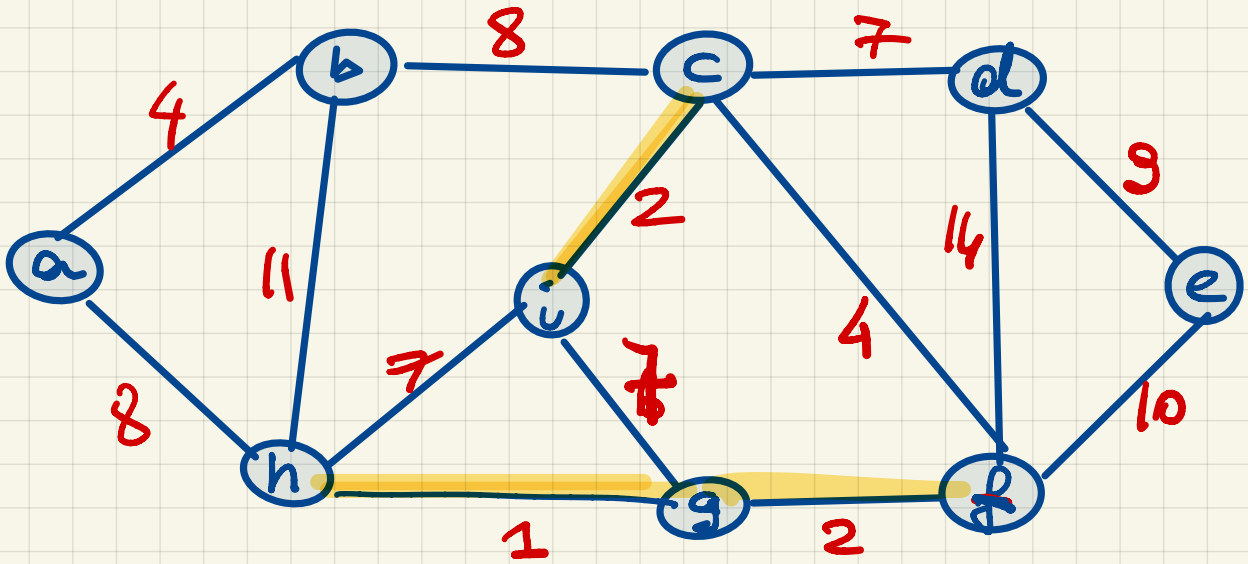
UNION(h, g)





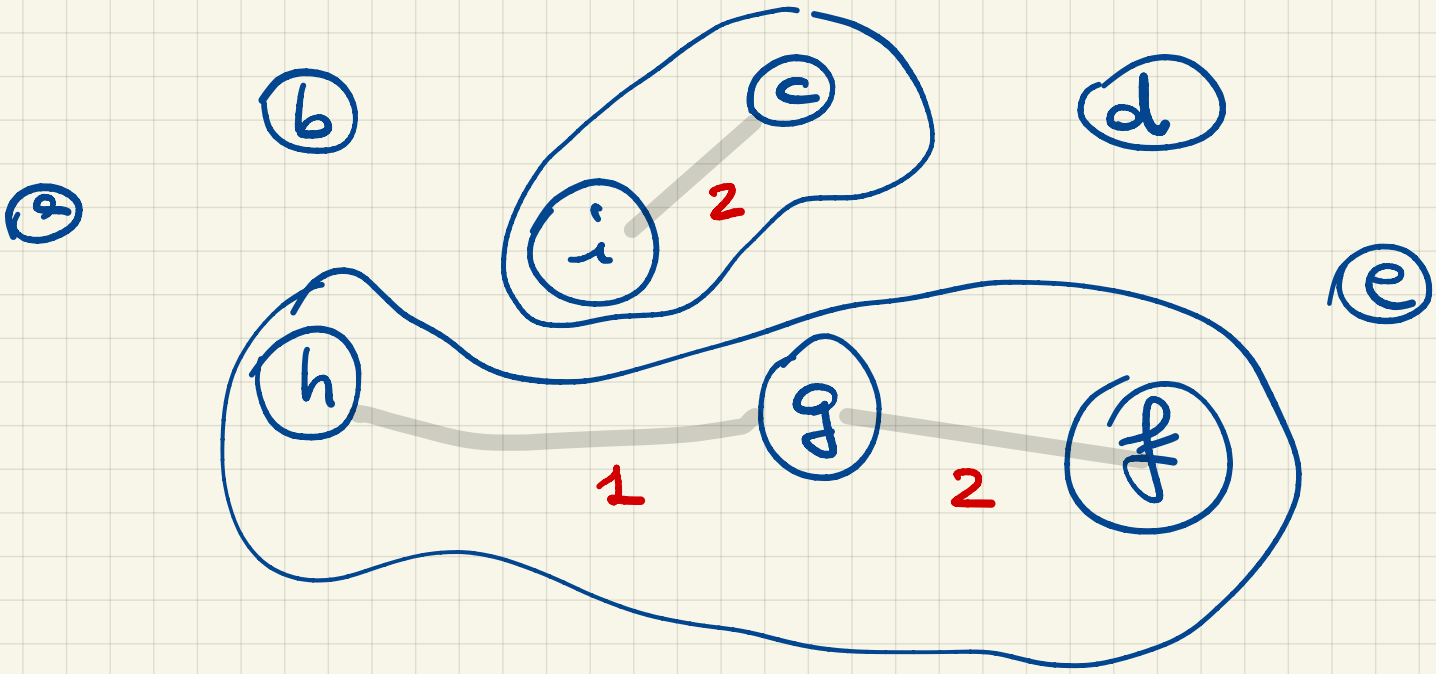
find_set (i) ≠ find_set (c)

UNION (i, c)

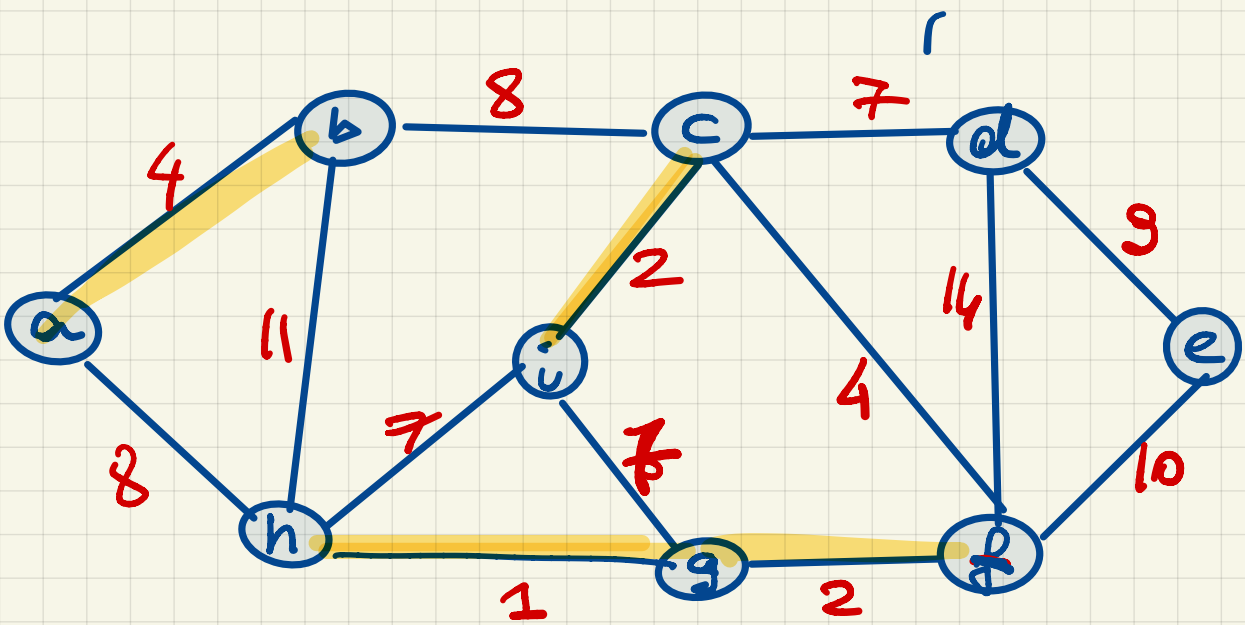


find_set (g) ≠ find_set (f)

UNION (g, f)

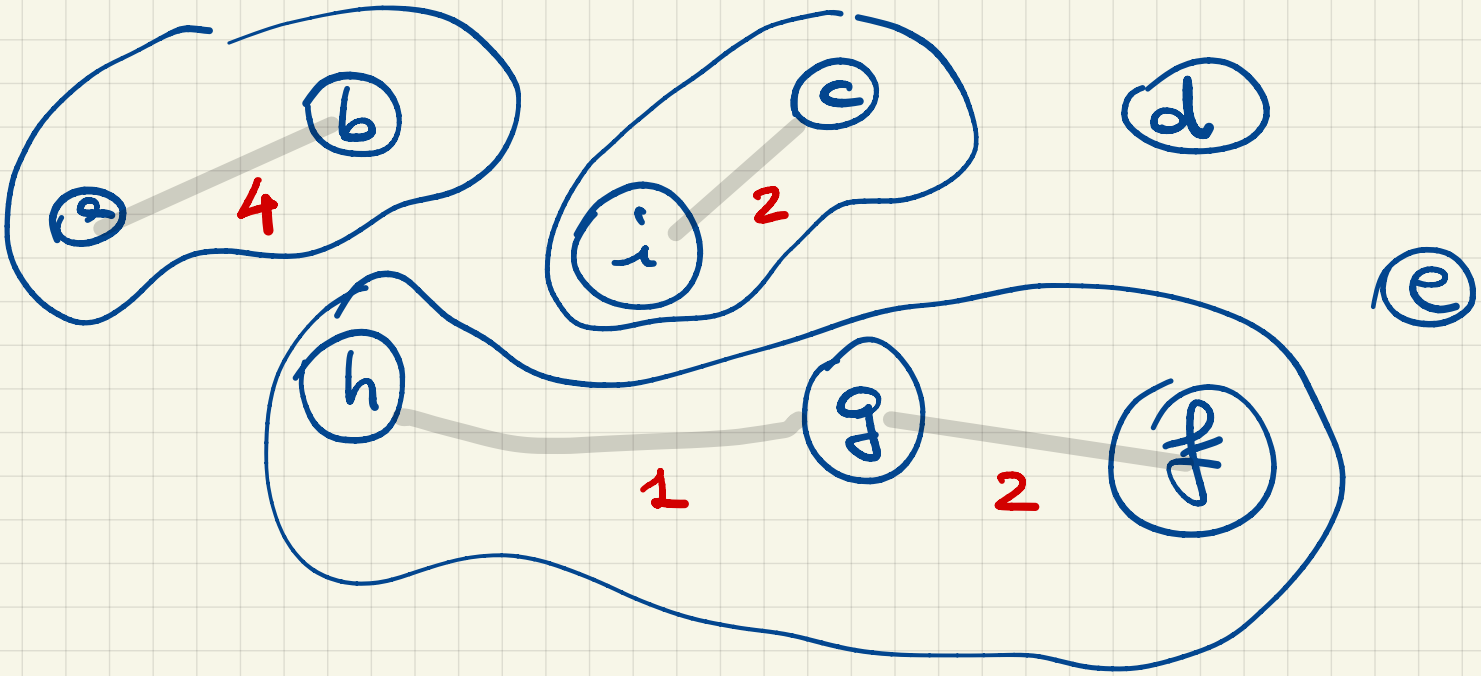


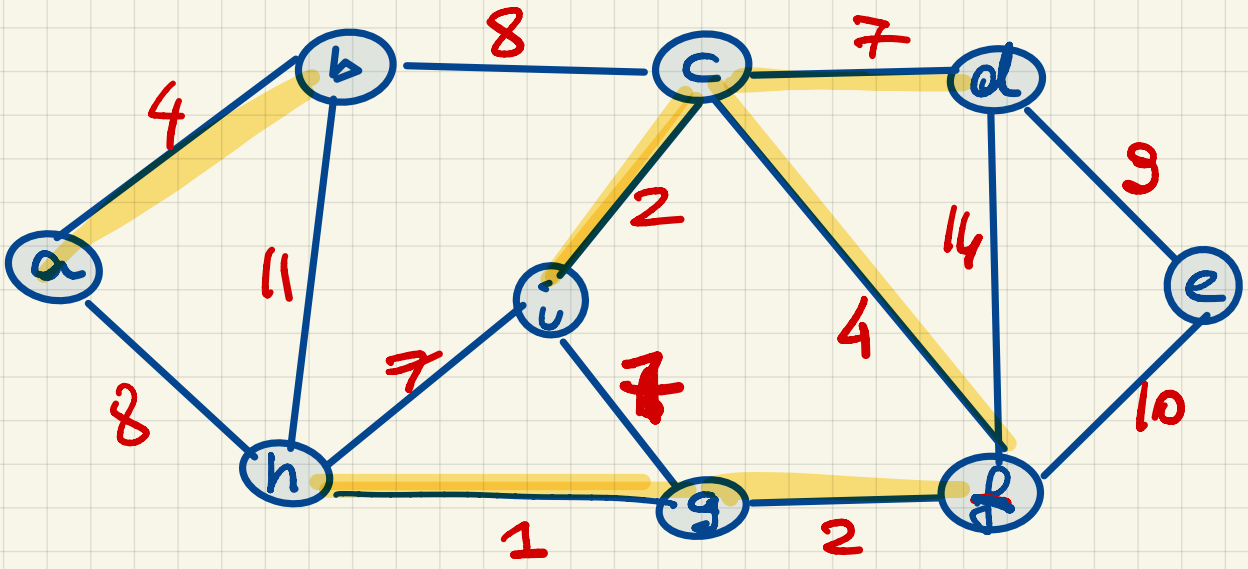
estraggo un'altro arco



find-set(a) ≠ find-set(b)

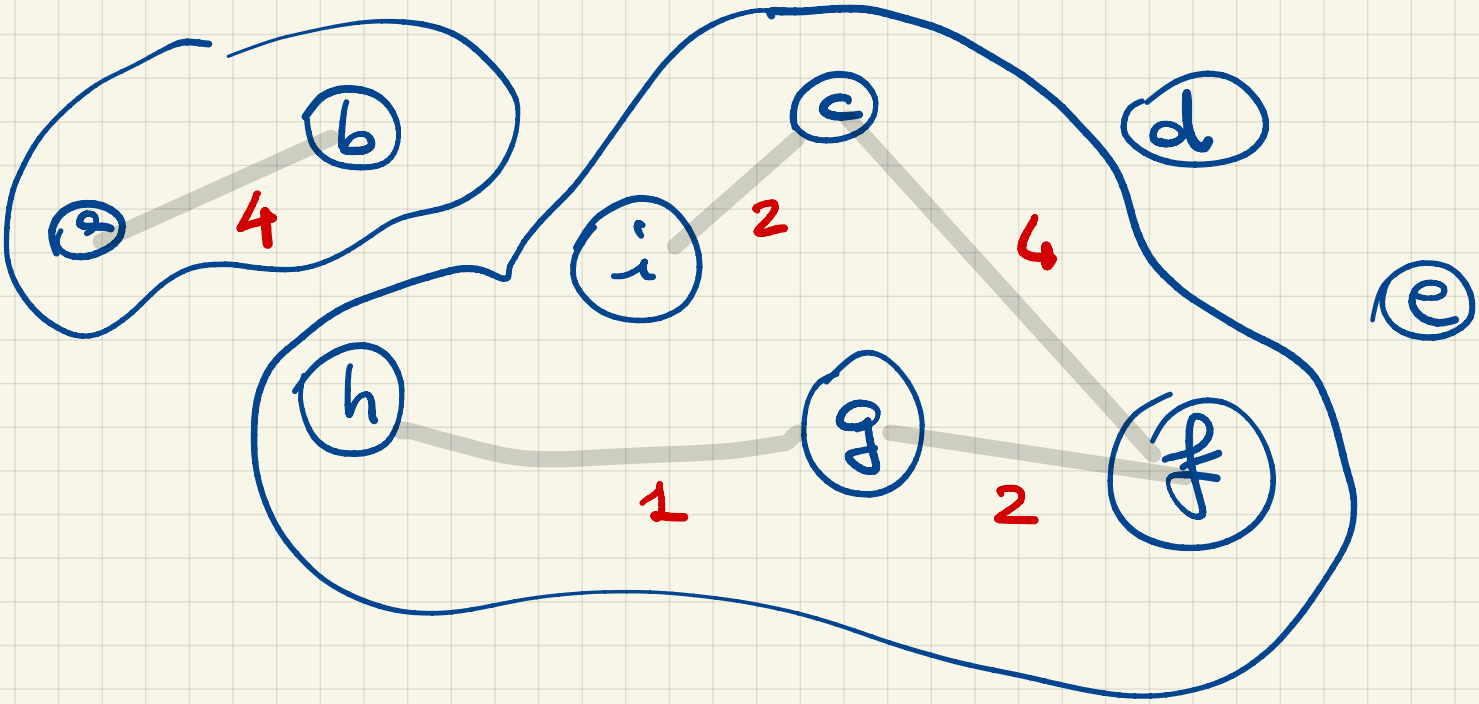
UNION(a, b)





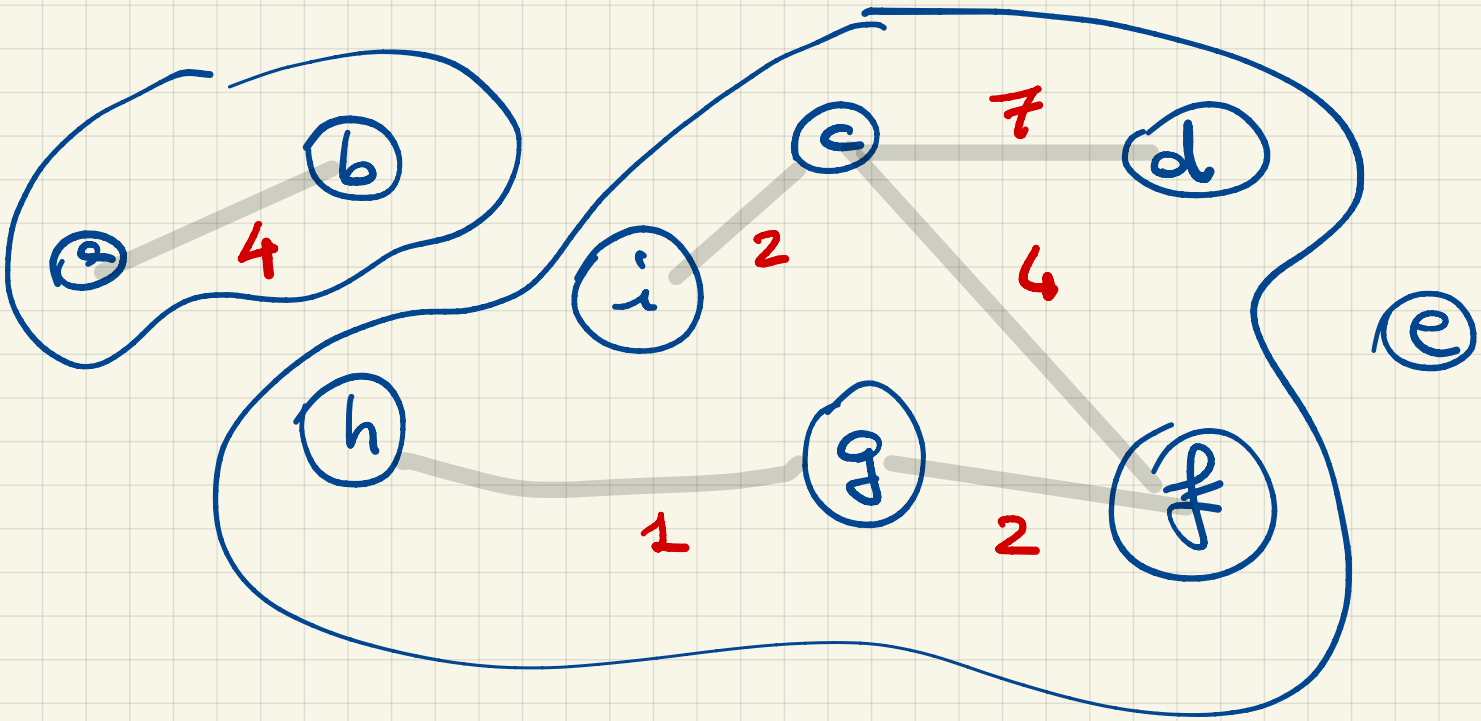
find-set(c) ≠ find-set(f)

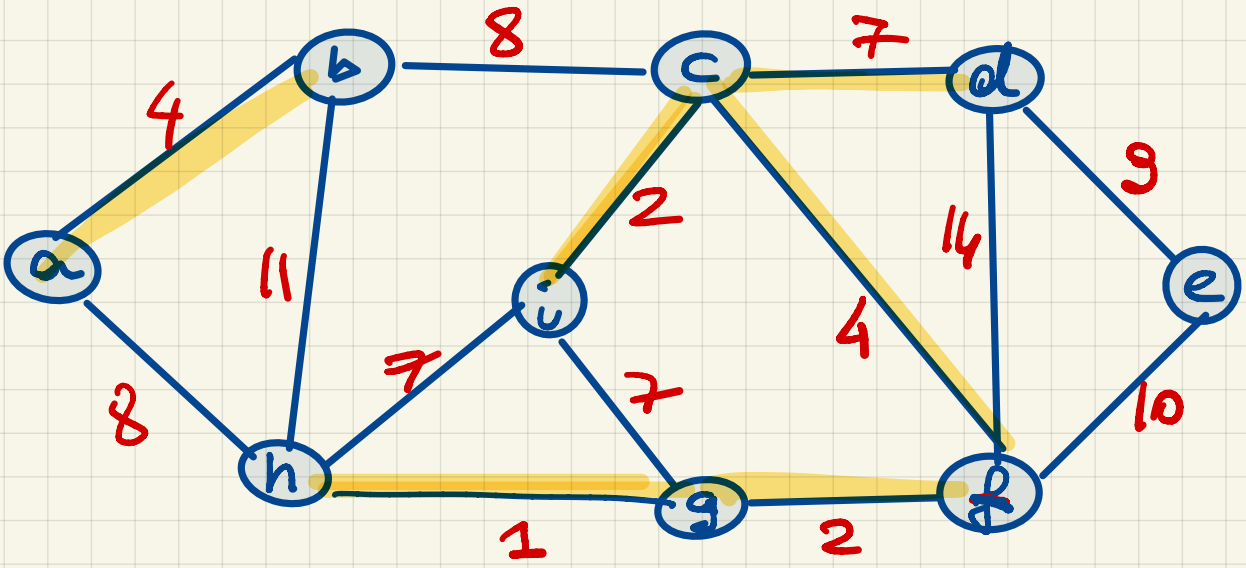
UNION(c, f)



selezionato l'arco (c, d)

posso fare UNION (c, d)

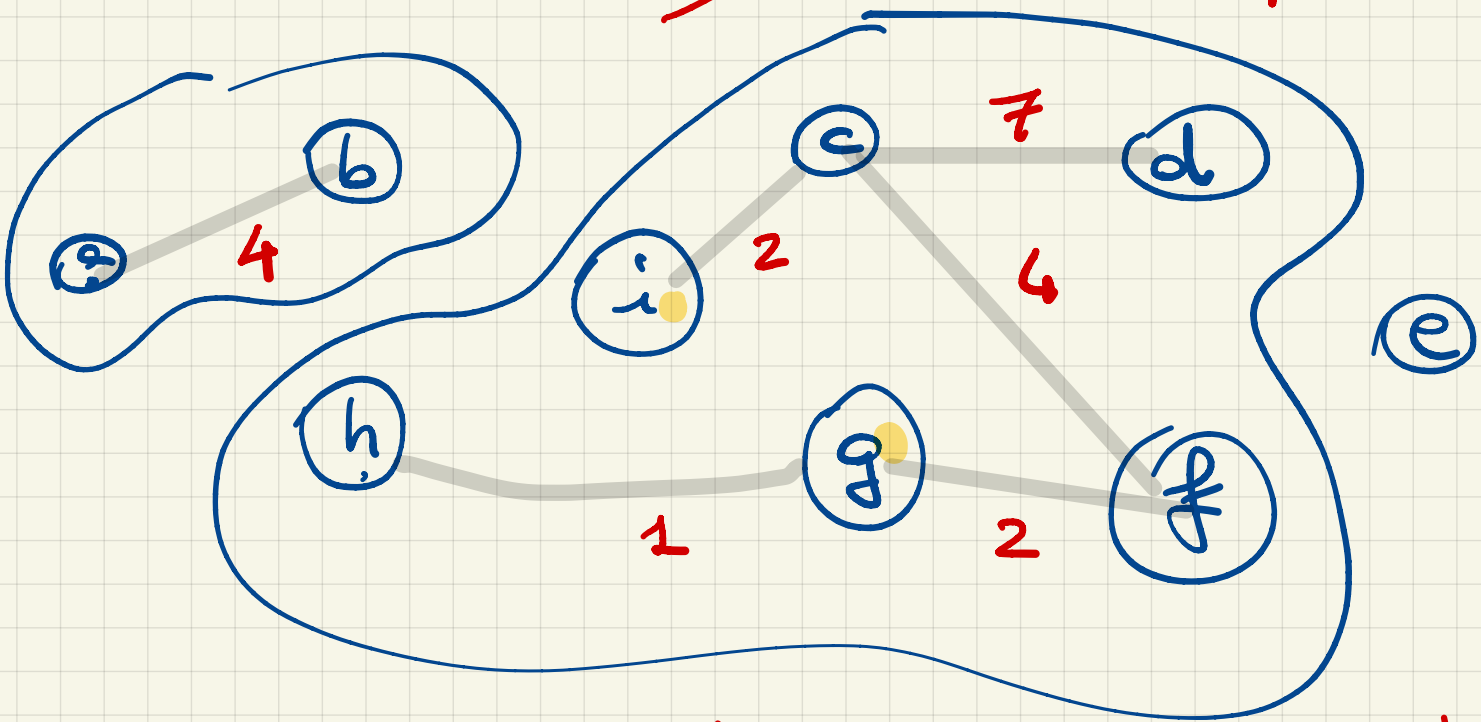




scelgo (i, g)

if find_set(i) ≠ find_set(g)

no



find_set(i) = find_set(g)

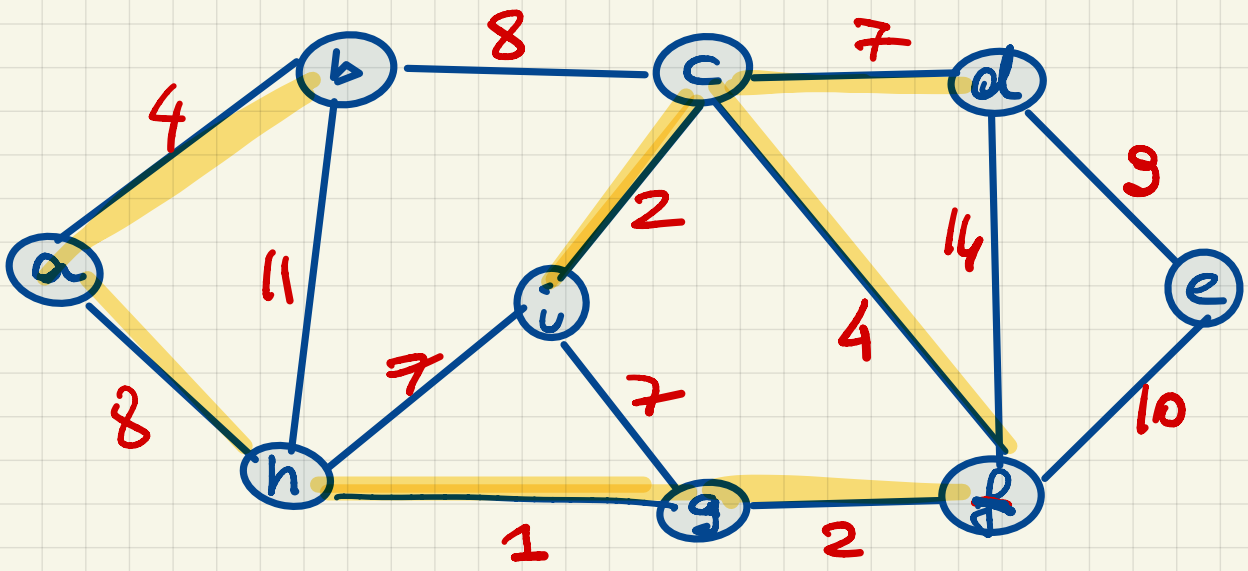
devo scegliere (h, i) ma

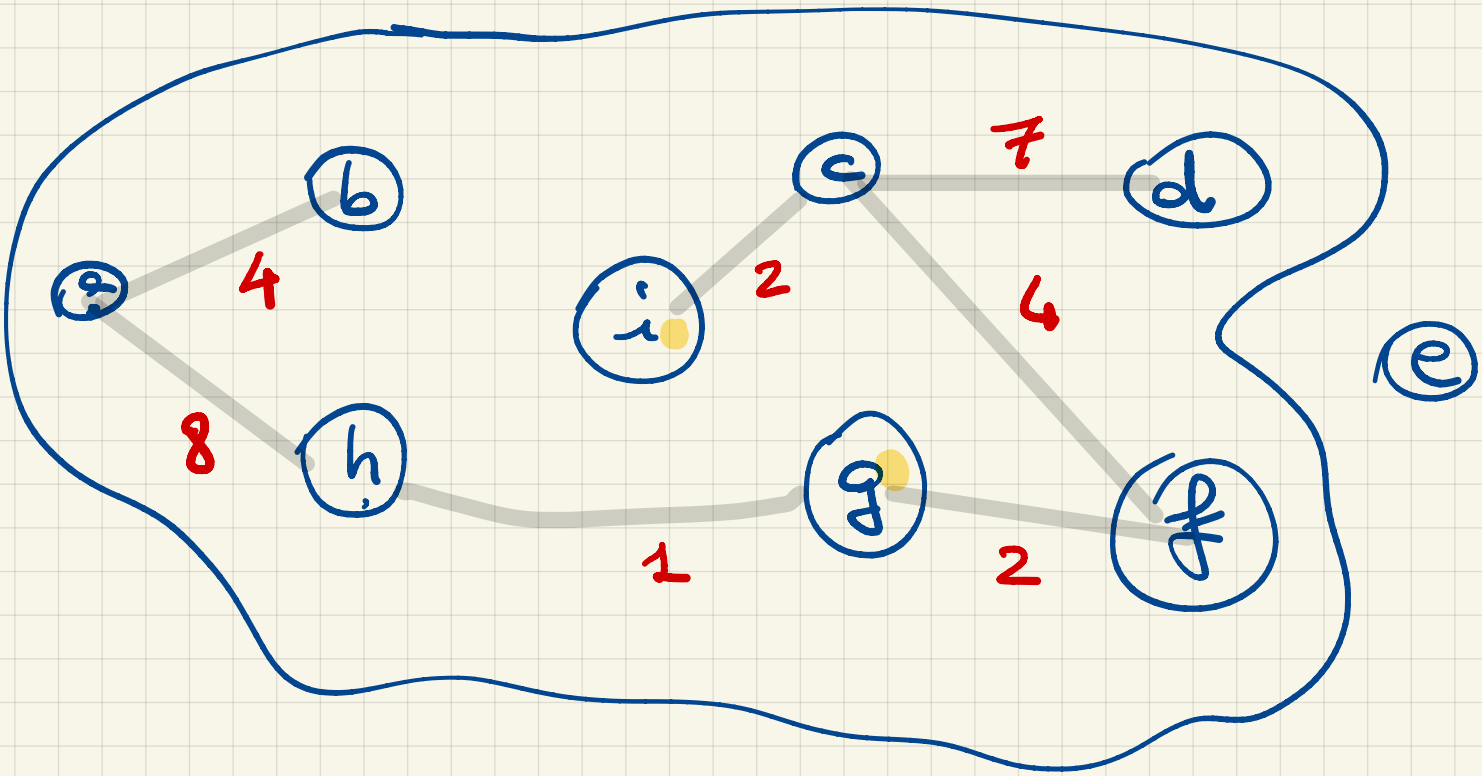
find_set(h) = find_set(i)
per cui b salta!

scelgo l'arco (a, h)

find_set(a) ≠ find_set(h)?

UNION(a, h)



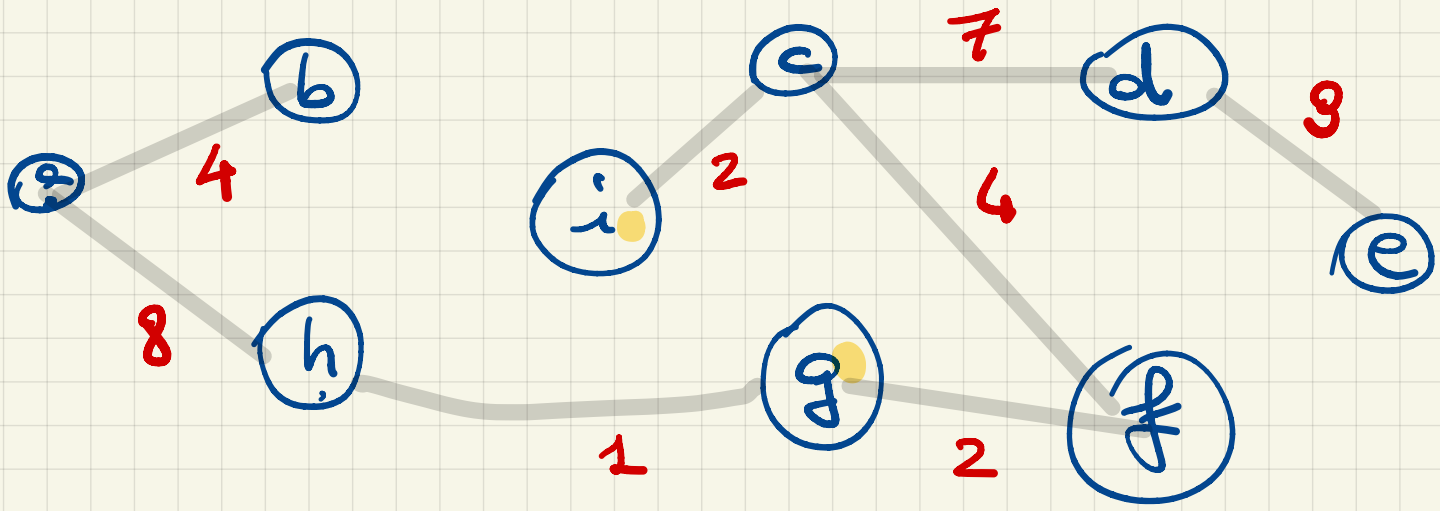


scelgo l'arco (b, c) di costo 8 però find-set(b) = find-set(c)

per cui lo salto!

scelgo l'arco (d, e) di costo 9, find-set(d) ≠ find-set(e)

per cui faccio UNION(e, d)



ottengo un'albero di copertura — Minimum Spanning Tree

la procedura KRUSKAL-MST continua!

N.B gli archi rimanenti vengono tutti scartati! per via della componente UNICA da cui è formata.